

Travaux Pratiques

Jeu du Pendu en Java

Formation : Programmation Orientée Objet en Java

Niveau : Débutant

Durée estimée : 4-6 heures

Table des matières

1. Introduction
2. Objectifs pédagogiques
3. Prérequis
4. Rappels théoriques
5. Phase 1 : Échauffement - Exercices de base
6. Phase 2 : Création de la classe GuessGame
7. Phase 3 : Création de la classe Main
8. Phase 4 : Amélioration et tests
9. Défis supplémentaires
10. Conclusion et ressources

1. Introduction

Bienvenue dans ce travaux pratiques sur le développement d'un jeu du Pendu en Java !
Ce TP a été conçu pour vous permettre d'apprendre par la pratique, en développant vous-même chaque partie du code.

Contrairement à un tutoriel classique où vous copiez simplement du code, ce TP vous guide étape par étape avec des explications théoriques et des exercices progressifs. Vous devrez réfléchir et coder par vous-même pour développer vos compétences en programmation.

 **IMPORTANT** : Ne cherchez pas à regarder la solution complète avant d'avoir essayé ! L'apprentissage vient de l'effort et des erreurs que vous ferez en chemin.

2. Objectifs pédagogiques

À la fin de ce TP, vous serez capable de :

- Créer et manipuler des classes en Java
- Utiliser les types de données primitifs et les objets
- Travailler avec les collections (List, Set)
- Implémenter des boucles (while, for, do-while)
- Utiliser les conditions (if, else)
- Comprendre et utiliser les modificateurs (final, private, public)
- Lire les entrées utilisateur avec Scanner
- Générer des nombres aléatoires avec Random
- Manipuler des chaînes de caractères (String)
- Structurer un projet Java de manière professionnelle

3. Prérequis

Avant de commencer, assurez-vous d'avoir :

- Java JDK installé (version 11 ou supérieure)
- Un IDE Java (IntelliJ IDEA, Eclipse, ou VS Code avec extension Java)
- Des connaissances de base en programmation (variables, conditions, boucles)
- De la motivation et de la patience !

4. Rappels théoriques

Avant de commencer le projet, révisons les concepts Java essentiels que nous utiliserons.

4.1 Les classes et les objets

Une classe est un modèle (template) qui définit les caractéristiques et comportements d'un type d'objet. Un objet est une instance d'une classe.

Exemple de classe :

```
public class Personne {  
    private String nom;    // Attribut  
    private int age;       // Attribut  
  
    public Personne(String nom, int age) {    // Constructeur  
        this.nom = nom;  
        this.age = age;  
    }  
  
    public void sePresenter() {    // Méthode  
        System.out.println("Je m'appelle " + nom);  
    }  
}
```

Utilisation :

```
Personne p = new Personne("Alice", 25);  
p.sePresenter();    // Affiche : Je m'appelle Alice
```

4.2 Les modificateurs d'accès et final

Les modificateurs d'accès :

- **public** : accessible depuis n'importe où
- **private** : accessible uniquement dans la classe
- **protected** : accessible dans la classe et ses sous-classes

Le modificateur final :

final : empêche la réassignation d'une variable. Pour les variables de types primitifs, cela rend la valeur constante. Pour les objets, cela empêche de changer la référence, mais l'objet lui-même reste mutable.

Exemple :

```
final int age = 25;
// age = 30; // ERREUR : impossible de réassigner

final List<String> noms = new ArrayList<>();
noms.add("Alice"); // OK : l'objet est mutable
// noms = new ArrayList<>(); // ERREUR : impossible de
réassigner
```

4.3 Les collections (List et Set)

List (ArrayList) : une liste ordonnée qui peut contenir des doublons

```
List<String> fruits = new ArrayList<>();
fruits.add("Pomme");
fruits.add("Banane");
fruits.add("Pomme"); // Doublon autorisé
System.out.println(fruits.get(0)); // Pomme
System.out.println(fruits.size()); // 3
```

Set (HashSet) : une collection non ordonnée qui ne peut pas contenir de doublons

```
Set<String> couleurs = new HashSet<>();
couleurs.add("Rouge");
couleurs.add("Bleu");
couleurs.add("Rouge"); // Ignoré (doublon)
System.out.println(couleurs.size()); // 2
```

4.4 Les boucles

Boucle while : répète un bloc tant qu'une condition est vraie

```
int i = 0;
while (i < 5) {
    System.out.println(i);
    i++;
}
```

Boucle for : répète un bloc un nombre précis de fois

```
for (int i = 0; i < 5; i++) {
    System.out.println(i);
}

// For-each pour parcourir une collection
List<String> noms = Arrays.asList("Alice", "Bob");
for (String nom : noms) {
    System.out.println(nom);
}
```

Boucle do-while : exécute le bloc au moins une fois, puis répète tant que la condition est vraie

```
int i = 0;
do {
    System.out.println(i);
    i++;
} while (i < 5);
```

4.5 Scanner et Random

Scanner : permet de lire les entrées utilisateur

```
Scanner scanner = new Scanner(System.in);
System.out.println("Entrez votre nom :");
String nom = scanner.nextLine();
System.out.println("Bonjour " + nom);
```

Random : permet de générer des nombres aléatoires

```
Random random = new Random();  
int nombre = random.nextInt(10); // 0 à 9  
int nombre2 = random.nextInt(5) + 1; // 1 à 5
```

4.6 Manipulation de chaînes de caractères (String)

Méthodes utiles de la classe String :

- `split(String regex)` : découpe une chaîne en tableau selon un séparateur
- `toCharArray()` : convertit une chaîne en tableau de caractères
- `charAt(int index)` : retourne le caractère à l'index donné
- `length()` : retourne la longueur de la chaîne
- `contains(CharSequence s)` : vérifie si la chaîne contient une sous-chaîne

Exemple :

```
String texte = "Bonjour le monde";  
String[] mots = texte.split(" "); // ["Bonjour", "le", "monde"]  
char premierCaractere = texte.charAt(0); // 'B'  
int longueur = texte.length(); // 16  
boolean contient = texte.contains("monde"); // true
```

5. Phase 1 : Échauffement - Exercices de base

Avant de commencer le projet principal, faisons quelques exercices pour vous échauffer. Créez un nouveau fichier Java appelé Exercices.java pour tester ces concepts.

Exercice 1 : Manipulation de listes

Objectif : Créer une liste de nombres et afficher ceux qui sont pairs.

Instructions :

1. Créez une ArrayList d'entiers
2. Ajoutez les nombres de 1 à 10
3. Parcourez la liste avec une boucle for-each
4. Affichez uniquement les nombres pairs (utilisez l'opérateur %)

 **Indice** : Un nombre est pair si $\text{nombre} \% 2 == 0$

 **À vous de jouer !** Essayez de coder cette solution avant de continuer.

Exercice 2 : Set et doublons

Objectif : Comprendre comment Set élimine les doublons.

Instructions :

5. Créez un HashSet de caractères
6. Ajoutez les lettres suivantes : 'a', 'b', 'a', 'c', 'b', 'd'
7. Affichez la taille du Set
8. Affichez le contenu du Set

Question : Combien d'éléments le Set contient-il et pourquoi ?

Exercice 3 : Scanner et validation

Objectif : Lire une entrée utilisateur et la valider.

Instructions :

9. Créez un Scanner
10. Demandez à l'utilisateur d'entrer une lettre
11. Vérifiez que l'entrée ne contient qu'un seul caractère
12. Si l'entrée est invalide, redemandez une lettre
13. Affichez la lettre validée

💡 **Indice** : Utilisez `input.length()` pour vérifier la longueur et une boucle `do-while` pour redemander

Exercice 4 : Random et tableaux

Objectif : Sélectionner un élément aléatoire dans un tableau.

Instructions :

14. Créez un tableau de String contenant 5 noms de fruits
15. Créez un objet Random
16. Générez un nombre aléatoire entre 0 et la taille du tableau - 1
17. Affichez le fruit sélectionné aléatoirement

💡 **Indice** : `random.nextInt(tableau.length)` génère un index valide

6. Phase 2 : Création de la classe GuessGame

Maintenant que vous êtes échauffé, passons au projet principal ! Nous allons créer le jeu du Pendu en commençant par la classe qui gère la logique du jeu.

6.1 Structure du projet

Créez la structure de dossiers suivante :

```
Pendu/
├── src/
│   ├── Main.java
│   └── com/
│       └── votrenom/
│           └── game/
│               └── GuessGame.java
└── README.md
```

 **Note** : Remplacez "votrenom" par votre prénom ou pseudonyme.

6.2 Étape 1 : Créer la classe GuessGame vide

Dans le fichier GuessGame.java, commencez par :

18. Déclarez le package (com.votrenom.game)
19. Importez les classes nécessaires : ArrayList, HashSet, List, Set
20. Créez une classe publique GuessGame
21. Ajoutez un commentaire JavaDoc pour décrire la classe

Exemple de structure :

```
package com.votrenom.game;

import java.util.ArrayList;
import java.util.HashSet;
import java.util.List;
import java.util.Set;

/**
 * Classe représentant le jeu du Pendu.
 */
public class GuessGame {
    // À compléter
}
```

6.3 Étape 2 : Définir les attributs de la classe

Notre jeu a besoin de stocker plusieurs informations. Réfléchissez à ce dont nous avons besoin :

- Le mot secret à deviner → Quel type de collection utiliser ?
- Le nombre de vies restantes → Quel type primitif ?
- Le mot avec les lettres devinées (avec des _ pour les lettres non trouvées) → Quelle collection ?
- Les lettres déjà essayées par le joueur → Quelle collection pour éviter les doublons ?

 **À vous de jouer !** Déclarez ces 4 attributs dans votre classe GuessGame. Pensez aux modificateurs d'accès (private) et final (quand c'est approprié).

 **Indices :**

- Pour le mot secret et le mot avec _, utilisez List<Character>
- Pour les vies, utilisez int (sans final car la valeur change)
- Pour les lettres essayées, utilisez Set<Character> (pas de doublons)
- Tous les attributs doivent être private

6.4 Étape 3 : Créer le constructeur

Le constructeur doit initialiser tous les attributs de la classe. Il prend en paramètres le mot secret (String) et le nombre de vies (int).

Le constructeur doit :

22. Convertir le mot secret (String) en List<Character>
23. Initialiser le nombre de vies
24. Remplir le mot à deviner avec des underscores (_)
25. Initialiser le Set des lettres essayées (il sera vide au début)

 **À vous de jouer !** Créez le constructeur public `GuessGame(String secretWord, int lifePoints)`

 **Indices :**

- Pour convertir un String en List<Character>, utilisez `toCharArray()` et une boucle `for-each`
- Pour remplir le mot à deviner avec des `_`, utilisez une boucle `for` avec `secretWord.length()`
- N'oubliez pas d'utiliser `"this."` pour faire référence aux attributs de la classe

Exemple de logique pour convertir un String en List :

```
for (char c : secretWord.toCharArray()) {  
    this.secretWord.add(c);  
}
```

6.5 Étape 4 : Créer la méthode `guessLetter`

C'est la méthode principale du jeu ! Elle vérifie si une lettre proposée par le joueur est dans le mot secret.

La méthode doit :

26. Recevoir un caractère (char) en paramètre
27. Ajouter la lettre au Set des lettres essayées
28. Vérifier si la lettre est dans le mot secret ET n'est pas déjà dans le mot deviné
29. Si la lettre est bonne : remplacer tous les `_` par la lettre aux bonnes positions
30. Si la lettre est mauvaise : retirer une vie

 **À vous de jouer !** Créez la méthode public void guessLetter(char letter)

 **Indices :**

- Utilisez secretWord.contains(letter) pour vérifier si la lettre est dans le mot
- Utilisez !guessWord.contains(letter) pour vérifier qu'elle n'est pas déjà trouvée
- Pour remplacer les _, parcourez secretWord avec un index et utilisez guessWord.set(index, letter)
- Pour retirer une vie, faites simplement lifePoints -= 1 ou lifePoints--

Algorithme suggéré :

1. Créer une variable boolean isGoodLetter
2. isGoodLetter = la lettre est dans secretWord ET pas dans guessWord
3. Ajouter la lettre au Set guessedLetters
4. Si isGoodLetter est true :
 - Parcourir secretWord avec un index
 - Pour chaque position où la lettre correspond, faire guessWord.set(index, letter)
5. Sinon :
 - Retirer une vie

6.6 Étape 5 : Créer les méthodes isWon() et isLost()

Ces méthodes permettent de vérifier si le jeu est terminé.

isLost() : retourne true si le joueur n'a plus de vies

 **À vous de jouer !** Créez la méthode public boolean isLost()

 **Indice :** Retournez simplement lifePoints <= 0

isWon() : retourne true si le joueur a trouvé toutes les lettres

 **À vous de jouer !** Créez la méthode public boolean isWon()

 **Indice :** Le jeu est gagné quand guessWord ne contient plus de '_'

6.7 Étape 6 : Surcharger la méthode toString()

La méthode toString() permet d'afficher l'état du jeu de manière lisible. Elle sera appelée automatiquement quand vous ferez System.out.println(game).

La méthode doit retourner une String contenant :

- Le mot à deviner (guessWord)
- Le nombre de vies restantes
- Les lettres déjà essayées

 **À vous de jouer !** Créez la méthode @Override public String toString()

Exemple de format souhaité :

```
mot à deviner : [c, o, _, _, r] | points de vie : 8 | lettres  
essayées : [a, b, c, o]
```

💡 **Indice** : Utilisez la concaténation de chaînes avec + et retournez le résultat

7. Phase 3 : Création de la classe Main

Maintenant que la logique du jeu est prête, créons la classe Main qui gère l'interaction avec le joueur.

7.1 Étape 1 : Structure de base du Main

Dans le fichier Main.java, créez :

31. Les imports nécessaires (Scanner, Random, GuessGame)
32. La classe publique Main
33. La méthode public static void main(String[] args)

```
import com.votrenom.game.GuessGame;
import java.util.Random;
import java.util.Scanner;

public class Main {
    public static void main(String[] args) {
        // À compléter
    }
}
```

7.2 Étape 2 : Initialisation des variables

Dans la méthode main, initialisez :

34. Un objet Random
35. Un tableau de mots (au moins 10 mots)
36. Le nombre de vies (par exemple 10)
37. Sélectionnez un mot aléatoire du tableau
38. Créez un objet GuessGame avec le mot et le nombre de vies

 **À vous de jouer !** Codez cette partie d'initialisation

 **Indices :**

- Pour créer le tableau facilement : "mot1 mot2 mot3".split(" ")
- Pour sélectionner aléatoirement : words[random.nextInt(words.length)]
- Utilisez final var pour des variables non réassignées

7.3 Étape 3 : La boucle principale du jeu

Créez une boucle infinie qui :

39. Affiche l'état du jeu (System.out.println(game))
40. Demande une lettre au joueur
41. Appelle game.guessLetter(letter)
42. Vérifie si le jeu est gagné ou perdu
43. Si le jeu est terminé, affiche le résultat et sort de la boucle

 **À vous de jouer !** Créez la boucle while(true) avec cette logique

 **Indice :** Utilisez break; pour sortir de la boucle quand le jeu est terminé

Structure suggérée :

```
while (true) {  
    // Afficher le jeu  
    // Lire la lettre  
    // Jouer la lettre  
    if (game.isLost()) {  
        // Afficher message de défaite  
    }  
    if (game.isWon()) {  
        // Afficher message de victoire  
    }  
    if (game.isWon() || game.isLost()) {  
        break;  
    }  
}
```

7.4 Étape 4 : Créer la méthode scanLetter

Pour rendre le code plus propre, créons une méthode qui lit et valide une lettre de l'utilisateur.

La méthode doit :

- 44. Être private static (car appelée depuis main)
- 45. Prendre une String question en paramètre
- 46. Retourner un char
- 47. Créer un Scanner
- 48. Redemander tant que l'entrée n'est pas valide (boucle do-while)
- 49. Vérifier que l'entrée contient exactement 1 caractère



À vous de jouer ! Créez la méthode private static char scanLetter(String question)



Indice : Utilisez scanner.nextLine() pour lire l'entrée, puis input.charAt(0) pour obtenir le premier caractère

Structure suggérée :

```
private static char scanLetter(String question) {
    Scanner scanner = new Scanner(System.in);
    Character letter = null;
    do {
        System.out.println(question);
        String input = scanner.nextLine();
        if (input.length() == 1) {
            letter = input.charAt(0);
        }
    } while (letter == null);
    return letter;
}
```

8. Phase 4 : Améliorations et tests

Maintenant que le jeu fonctionne, ajoutons quelques améliorations !

8.1 Amélioration 1 : Permettre de rejouer

Modifiez votre boucle principale pour :

- 50. Après la fin d'une partie, demander au joueur s'il veut rejouer
- 51. Si oui, recommencer une nouvelle partie
- 52. Si non, quitter le programme



À vous de jouer ! Implémentez cette fonctionnalité



Indice : Utilisez `scanLetter` pour lire la réponse (y/n), puis vérifiez si c'est 'y', 'Y', 'o' ou 'O'

8.2 Amélioration 2 : Meilleure validation des entrées

Améliorez la méthode `scanLetter` pour :

- 53. Accepter uniquement les lettres (pas de chiffres ou caractères spéciaux)
- 54. Convertir automatiquement en minuscule
- 55. Afficher un message d'erreur si l'entrée est invalide



Indice : Utilisez `Character.isLetter(char)` pour vérifier si c'est une lettre

8.3 Tests du jeu

Testez votre jeu avec différents scénarios :

- ✓ Gagner une partie en trouvant toutes les lettres
- ✓ Perdre une partie en épuisant toutes les vies
- ✓ Essayer une lettre déjà proposée
- ✓ Entrer des caractères invalides
- ✓ Rejouer après une partie

9. Défis supplémentaires

Vous avez terminé le projet de base ? Voici quelques défis pour aller plus loin !

Défi 1 : Affichage du pendu visuel

Créez une méthode qui affiche un dessin ASCII du pendu qui évolue en fonction du nombre de vies restantes.

Exemple :

```
+---+
|   |
O   |
/|\  |
/ \  |
     |
=====
```

Défi 2 : Système de difficulté

Ajoutez différents niveaux de difficulté :

- Facile : 15 vies, mots courts (4-6 lettres)
- Moyen : 10 vies, mots moyens (7-9 lettres)
- Difficile : 7 vies, mots longs (10+ lettres)

Défi 3 : Système de score

Implémentez un système de score qui :

- Donne des points pour chaque lettre correcte
- Retire des points pour chaque erreur
- Donne un bonus si le mot est trouvé rapidement
- Sauvegarde le meilleur score

Défi 4 : Catégories de mots

Organisez vos mots par catégories (animaux, pays, fruits, etc.) et laissez le joueur choisir une catégorie avant de commencer.

Défi 5 : Sauvegarde des parties

Permettez de sauvegarder une partie en cours dans un fichier et de la charger plus tard pour continuer.

10. Conclusion et ressources

Félicitations !



Vous avez terminé le projet du jeu du Pendu ! Vous avez maintenant une bonne compréhension des concepts fondamentaux de la programmation Java.

Ce que vous avez appris

- ✓ Créer et organiser des classes Java
- ✓ Utiliser les collections (List, Set)
- ✓ Implémenter des boucles et conditions
- ✓ Gérer les entrées utilisateur
- ✓ Structurer un projet avec plusieurs classes
- ✓ Appliquer les principes de la POO

Prochaines étapes

Pour continuer votre apprentissage :

- 56. Explorez les concepts avancés : héritage, interfaces, exceptions
- 57. Apprenez à utiliser les streams et lambdas (Java 8+)
- 58. Découvrez les frameworks populaires : Spring Boot, JavaFX
- 59. Pratiquez avec d'autres projets : calculatrice, jeu de morpion, gestionnaire de tâches
- 60. Contribuez à des projets open source sur GitHub

Ressources utiles

Documentation officielle Java :

- <https://docs.oracle.com/en/java/>

Tutoriels et exercices :

- <https://www.codecademy.com/learn/learn-java>
- <https://www.jetbrains.com/academy/>

Bon courage dans votre parcours d'apprentissage Java !

 Happy Coding! 